Министерство цифрового развития, связи и массовых коммуникаций.
Федеральное государственное бюджетное учреждение высшего образования «Сибирский государственный университет телекоммуникаций и информатики» Уральский технический институт связи и информатики (филиал) в г. Екатеринбурге
Цикловая комиссия ИТ и АСУ кафедры ИСТ

Отчет
по учебной практике
по МДК 02.02 «Инструментальные средства разработки программного обеспечения»

студента III курса 382 группы
Фамилия Арнаутовой
Имя, отчество Екатерины Дмитриевны
Факультет Инфокоммуникаций, информатики и управления
Специальность 09.02.07 «Основы алгоритмизации и программирования»

Екатеринбург, 2026

Содержание

Введение 3
1. Используемы технологии 4
2. Проектирование базы данных 5
2.1 Анализ предметной области 5
2.2 ER-диаграмма и нормализация 5
3. Разработка серверной части (Web API) 6
3.1 Структура проекта 6
3.2 Модели данных 7
3.3 Контроллеры 8
4. Разработка клиентской части (HTML/CSS/JavaScript) 9
4.2 Взаимодействие с API 10
4.3 Внешний вид интерфейсов 12
5 Описание практической части дипломного проекта 14
5.1 Структурная схема программы 14
5.2 Проектирование базы данных 15
5.3 Практическая реализация приложения 19
5.4 Инструкция по работе с программой 21
Список использованных источников 27

Введение

Задачи, решаемые в ходе учебной практики, включали разработку двух проектов, направленных на автоматизацию деятельности компании «Наш декор». Первая часть работы была посвящена созданию подсистемы учета продукции, которая позволяет просматривать, добавлять и редактировать данные о продукции и материалах, а также рассчитывать стоимость продукции, исходя из используемых материалов. Данная подсистема разрабатывалась как задание демонстрационного экзамена по специальности 09.02.07 «Информационные системы и программирование». Исходными данными послужили таблицы Excel с номенклатурой и руководство по стилю оформления интерфейса (Приложение 2).

Вторая часть работы является составной частью выпускной квалификационной работы и направлена на разработку автоматизированной системы учета заявок на установку, наладку и ремонт КИПиА.

Цель практики — закрепление навыков разработки информационных систем на примере экзаменационного задания и подготовка полноценного описания практической части дипломного проекта.

1. Используемы технологии

Для реализации проекта был выбран следующий стек технологий:

- среда разработки: Microsoft Visual Studio 2022 Community Edition;
- система контроля версий: Git (локальный репозиторий с возможностью публикации на удалённый сервер);
- языки программирования: C# (backend), JavaScript (frontend), SQL;
- платформа для backend: ASP.NET Core 8 (Web API);
- ORM: Entity Framework Core 8;
- СУБД: PostgreSQL (через pgAdmin 4) – для хранения данных продукции и материалов;
- клиентская часть: чистый HTML5, CSS3, JavaScript;
- работа с HTTP: Fetch API;
- инструменты тестирования: Postman (для проверки API), браузерные инструменты разработчика;
- Entity Framework Core – взаимодействие с базой данных, миграции;
- LINQ – для выборок и расчёта стоимости продукции.

2. Проектирование базы данных

2.1 Анализ предметной области

Исходными данными для проектирования послужили файлы импорта, на основании которых были выделены ключевые сущности предметной области.

Центральной сущностью является «Продукция» (Product) — обои, которые характеризуются артикулом, названием, ценой для партнера, шириной и принадлежностью к определенному типу. Справочник «Типы продукции» (ProductType) хранит классификаторы и коэффициенты, влияющие на расчетные показатели.

Производственная номенклатура сырья представлена сущностью «Материал» (Material) с полным набором учетных характеристик: цена, остатки на складе, единицы измерения и тип материала. Сами категории сырья (бумага, краска и т.д.) с указанием процента брака вынесены в отдельный справочник — «Тип материала» (MaterialType).

Связь между продукцией и материалами реализована через сущность «Состав продукции» (ProductMaterial), которая хранит нормы расхода каждого материала на единицу продукции, обеспечивая тем самым возможность расчета себестоимости.

2.2 ER-диаграмма и нормализация

База данных спроектирована в третьей нормальной форме. Все таблицы имеют первичные ключи, внешние ключи обеспечивают ссылочную целостность.

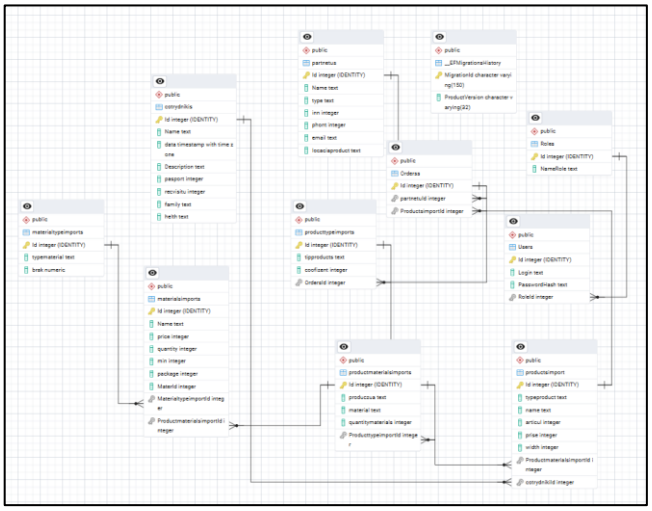

Рисунок 2.1 – ER-диаграмма БД

3. Разработка серверной части (Web API)

3.1 Структура проекта

В качестве технической платформы выбрана ASP.NET Core Web API. Разработка базы данных велась с использованием подхода Code First: на основании разработанных моделей данных и класса контекста посредством миграций Entity Framework Core выполнено автоматическое создание структуры таблиц. Уровень контроллеров содержит реализацию основных CRUD-методов для управления сущностями. Клиентский интерфейс построен на технологиях HTML/CSS, а для обращения к серверным методам API используются JavaScript-вызовы.

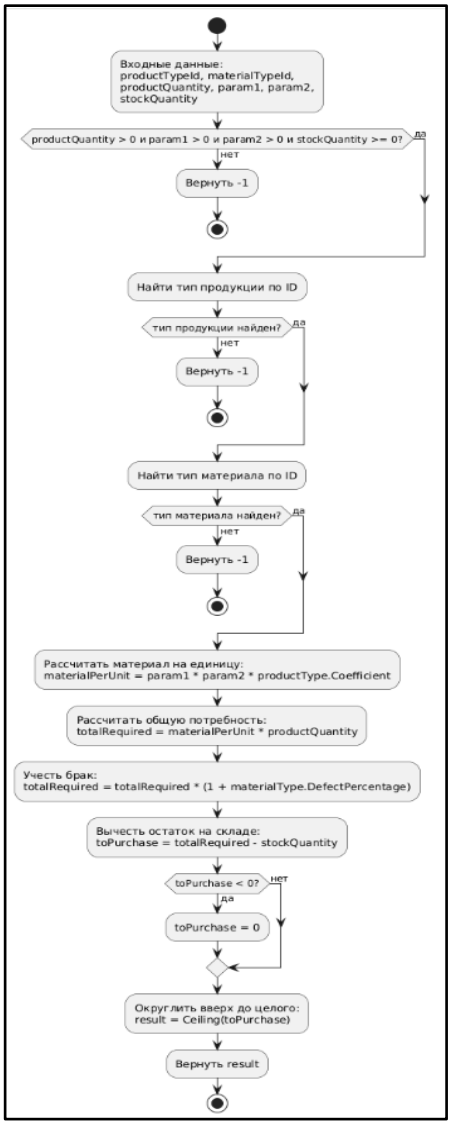

Рисунок 3.1 – Блок-схема структуры приложения

3.2 Модели данных

Созданы классы, соответствующие таблицам. Пример модели Product на рисунке 3.2.

```
namespace WebApplication10.model
{
    Ссылок: 3
    public class Producttypeimport
    {
        Ссылок: 0
        public int Id { get; set; }
        Ссылок: 0
        public string tipproducts { get; set; }//Тип продукции

        Ссылок: 0
        public int coofizent { get; set; } // Коэффициент типа продукции
        Ссылок: 1
        public List<Productmaterialsimport> productmaterialsimports { get; set; }
        Ссылок: 1
        public int OrdersId { get; set; }
        Ссылок: 1
        public Orders Orderss { get; set; }
    }
}
```

Рисунок 3.2 – Модель продукции

В контексте БД были добавлены все созданные модели и были настроены связи и настройки в методе onmodelcreating.

```
public class AppDbContext : DbContext
{

    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }


    public DbSet<Role> Roles { get; set; }

    public DbSet<User> Users { get; set; }

    public DbSet<Orders> Orderss { get; set; }

    public DbSet<partnetu> partnetus { get; set; }

    public DbSet<cotrydniki> cotrydnikis { get; set; }

    public DbSet<Productsimport> productsimport { get; set; }
    Ссылок: 0
    public DbSet<Productmaterialsimport> productmaterialsimports { get; set; }
    Ссылок: 0
    public DbSet<Producttypeimport> producttypeimports { get; set; }

    Ссылок: 5
    public DbSet<Materialsimport> materialsimports { get; set; }
    Ссылок: 0
    public DbSet<Materialtypeimport> materialtypeimports { get; set; }


    Ссылок: 0
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
```

Рисунок 3.3 – Контекст БД

3.3 Контроллеры

Серверная часть приложения включает три ключевых контроллера: ProductsController, MaterialsController и ValuesController (аутентификация и регистрация пользователей). Для каждого из них, кроме контроллера авторизации, реализован полный спектр CRUD-операций, следующих архитектурным принципам REST.

Набор методов для работы с продукцией (/api/products):

– GET (без параметров) — возвращает список всех продуктов с вычислением стоимости на основе связанных материалов;

– GET (с {id}) — выводит подробные сведения об указанном продукте;

– POST — создает новую запись о продукте;

– PUT (с {id}) — вносит изменения в существующий продукт;

– DELETE (с {id}) — удаляет запись о продукте.

Аналогичная структура эндпоинтов с соответствующими маршрутами реализована в контроллере MaterialsController для обработки данных о материалах.

Для хеширования паролей пользователей в базе данных в контроллере авторизации и регистрации была задействована библиотека BCrypt. На рисунке 3.4 представлен класс, содержащий методы, используемые данным контроллером.

```
public static class PasswordHelper
{
    Ссылок: 1
    public static string CreatePasswordHash(string password) =>
        BCrypt.Net.BCrypt.HashPassword(password);

    Ссылок: 1
    public static bool VerifyPassword(string password, string hash) =>
        BCrypt.Net.BCrypt.Verify(password, hash);
}
```

Рисунок 3.4 – Класс для хеширования и проверки пароля

4. Разработка клиентской части (HTML/CSS/JavaScript)

Оформление разработанных экранных форм выполнено в строгом соответствии с руководством по стилю, изложенным в Приложении 2. Визуальные компоненты системы подчинены следующим требованиям:

Шрифт: в качестве основного шрифта для всех элементов интерфейса используется гарнитура Gabriola.

Цветовая схема: основной фон страниц — белый (#FFFFFF). Для выделения шапки, заголовков таблиц и отдельных блоков применяется дополнительный фон #BBD9B2. Акцентирующий цвет #2D6033 используется для заголовков, кнопок сохранения, гиперссылок и индикации границ при фокусе элемента.

Графические элементы: в верхней части каждой страницы размещен логотип (файл logo.png) с сохранением исходных пропорций изображения. Для всех страниц установлена стандартная иконка сайта (favicon.ico).

```
<meta charset="utf-8">
<title>Products</title>
<style>
    body {
        font-family: 'Gabriola', serif;
        background-color: #FFFFFF;
        margin: 0;
        padding: 20px;
        color: #333;
    }

    /* Шапка с логотипом */
    .header {
        display: flex;
        align-items: center;
        background-color: #BBD9B2;
        padding: 15px 20px;
        border-radius: 8px;
        margin-bottom: 30px;
    }

    .logo {
        height: 60px;
        width: auto;
        margin-right: 20px;
    }

    .header h1 {
        color: #2D6033;
        font-size: 2.2em;
        margin: 0;
    }
```

Рисунок 3.5 – Основные используемы стили на веб-страницах

4.2 Взаимодействие с API

На каждой веб-странице используются конечные методы для отправки запросов к API. Методы были сделаны на основе сгенерированного кода через Postman.

```
async function loadProducts() {

    const response = await fetch(api + "/get");
    const data = await response.json();

    let rows = "";

    data.forEach(p => {

        rows += `
        <tr>
            <td>${p.id}</td>
            <td>${p.typeproduct}</td>
            <td>${p.name}</td>
            <td>${p.articul}</td>
            <td>${p.prise}</td>
            <td>${p.width}</td>
            <td>
                <button onclick="editProduct(${p.id})">Edit</button>
            </td>
        </tr>
        `;

    });

    document.getElementById("productsTable").innerHTML = rows;
}
```

Рисунок 3.6 – Эндпоинт для get

```
async function addProduct() {

    const product = {

        typeproduct: document.getElementById("typeproduct").value,
        name: document.getElementById("name").value,
        articul: parseInt(document.getElementById("articul").value),
        prise: parseInt(document.getElementById("prise").value),
        width: parseInt(document.getElementById("width").value)

    };

    await fetch(api + "/Post", {

        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(product)

    });

    loadProducts();
}
```

Рисунок 3.7 –Эндпоинт для post

```
async function updateProduct() {

    const id = document.getElementById("id").value;

    const product = {

        id: parseInt(id),
        typeproduct: document.getElementById("typeproduct").value,
        name: document.getElementById("name").value,
        articul: parseInt(document.getElementById("articul").value),
        prise: parseInt(document.getElementById("prise").value),
        width: parseInt(document.getElementById("width").value)

    };

    await fetch(api + "/Put/" + id, {

        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(product)

    });

    loadProducts();
}
```

Рисунок 3.8 – Эндпоинт для put

```
async function deleteProduct() {

    const id = document.getElementById("deleteId").value;

    await fetch(api + "/Delete/" + id, {
        method: "DELETE"
    });

    loadProducts();
}
```

Рисунок 3.9 – Эндпоинт для delete

На рисунках 3.6 – 3.9 отображены основные функций для взаимодействия с методами GET, POST, PUT, DELETE контроллеров, с помощью этих функций данные для вывода или ввода берутся из элементов страницы и передаются в методы.

4.3 Внешний вид интерфейсов

Рисунок 4.1 – Страница регистрации

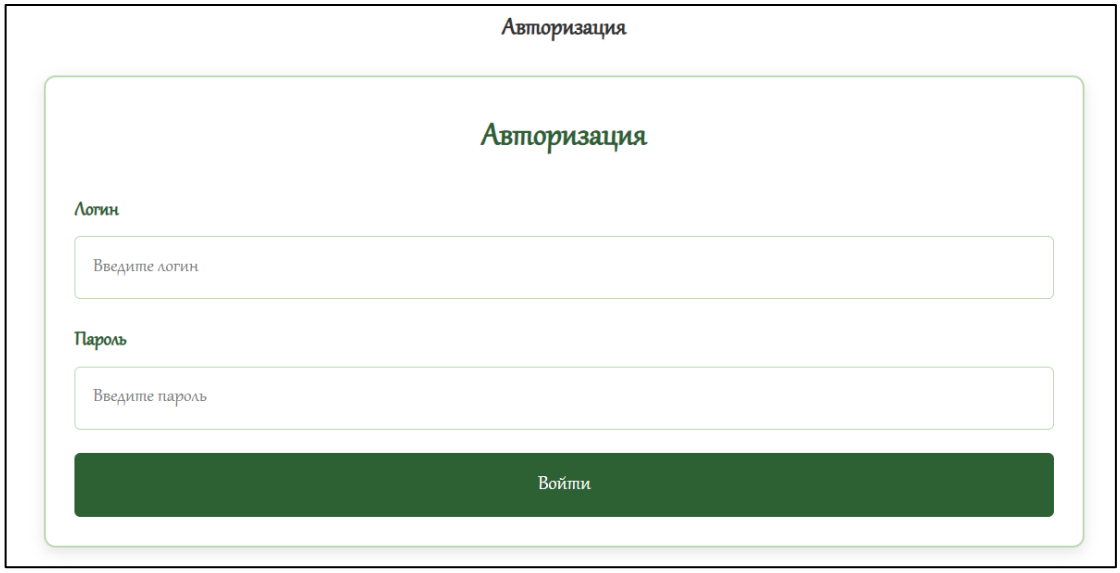

Рисунок 4.2 – Страница авторизации

Рисунок 4.3 – Страница продукции Products.html

На рисунке 4.3 показан страница с отображением продукции, в колонке с действиями при нажатии на кнопку “Edit”, на перенесет на форму с где мы можем добавить, изменить или удалить запись, форма показана на рисунке 4.4.

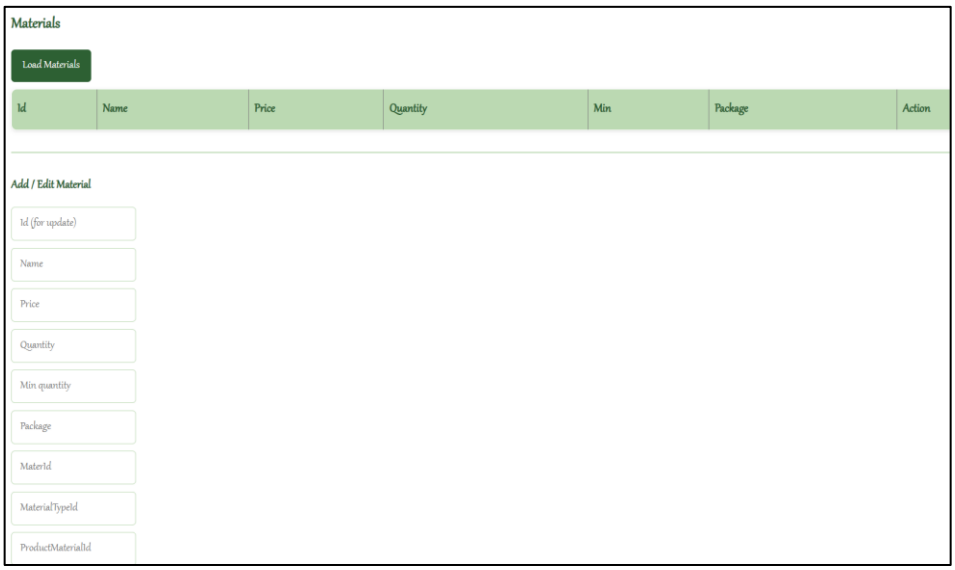

Рисунок 4.4 – Страница для просмотра и взаимодействия с материалами

5 Описание практической части дипломного проекта

5.1 Структурная схема программы

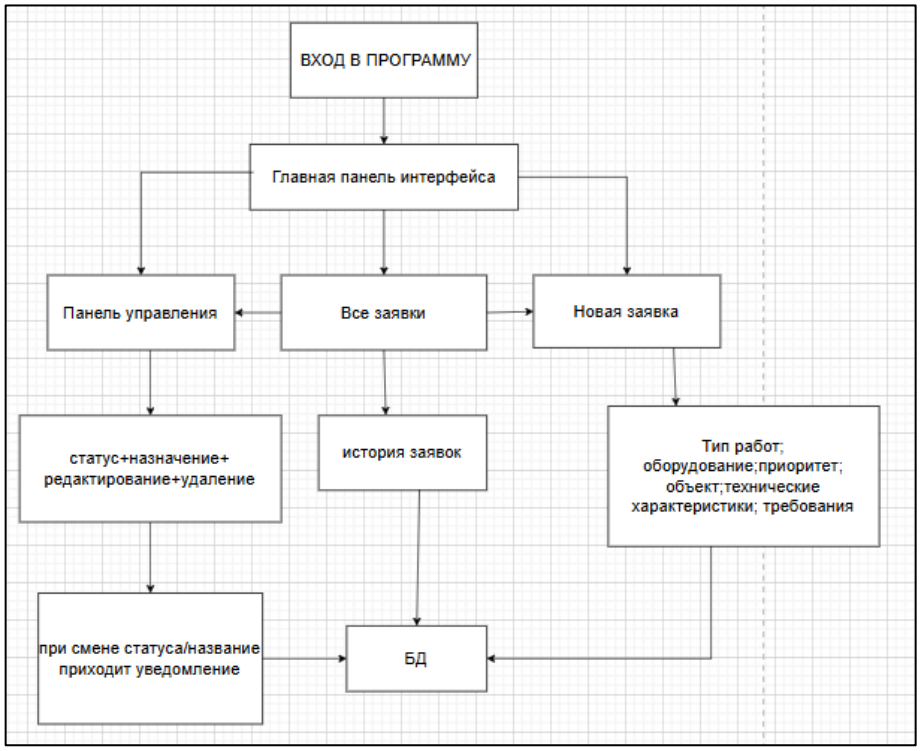

Рисунок 5.1 — Структурная схема

На рисунке 5.1 изображена общая структура приложения с взаимодействием отдельных компонентов между собой.

5.2 Проектирование базы данных

5.2.1 Содержание базы данных

1. Таблица: Equipment (Оборудование)
Id – уникальный идентификатор оборудования (первичный ключ)
Name – тип оборудования КИПиА
Description – описание ёоборудования
2. Таблица: ObjectEntity (Объекты)
Id – уникальный идентификатор объекта (первичный ключ)
Name – название объекта (например, ГРС)
Location – местоположение объекта
3. Таблица: User (Пользователи)
Id – уникальный идентификатор пользователя
Username – имя пользователя
PasswordHash – хэш пароля
Email – электронная почта для уведомлений
Role – роль пользователя (Applicant, Executor, Dispatcher, Admin)
4. Таблица: Request (Заявки)
Id –уникальный идентификатор заявки (первичный ключ)
UniqueNumber – уникальный номер заявки (автогенерация GUID)
CreatedAt – дата создания заявки
WorkType – тип работы
Priority – приоритет
Status – статус заявки
TechnicalSpecs – технические характеристики
Requirements – требования
EquipmentId – внешний ключ к таблице Equipment
ObjectId – внешний ключ к таблице ObjectEntity
AssigneeId – внешний ключ к таблице User (исполнитель, может быть NULL)
CreatorId – внешний ключ к таблице User (создатель заявки)
Таблица: RefreshToken (Токены обновления)
Id – уникальный идентификатор токена (первичный ключ)
UserId – внешний ключ к таблице User
Token – токен обновления
Expires – срок действия токена
IsRevoked – флаг отзыва токена
Таблица: UserSession (Сессии пользователей)
Id – уникальный идентификатор сессии (первичный ключ)
UserId – внешний ключ к таблице User
Token – токен сессии

CreatedAt – дата создания сессии
ExpiresAt – срок действия сессии
IsActive – флаг активности сессии
IpAddress – IP-адрес пользователя
UserAgent – информация о браузере и устройстве пользователя
Связи между таблицами:

- таблица Request связана с таблицей Equipment через EquipmentId;
- таблица Request связана с таблицей ObjectEntity через ObjectId;
- таблица Request связана с таблицей User через CreatorId (создатель) и AssigneeId (исполнитель);
- таблица RefreshToken связана с таблицей User через UserId;
- таблица UserSession связана с таблицей User через UserId.

5.2.2 Основные отношения

Отношение «один-ко-многим» между пользователями и заявками.

– Один пользователь (роли: Applicant, Dispatcher, Admin) может создавать множество заявок через связь CreatorId.

– Один пользователь (роли: Executor, Dispatcher, Admin) может быть назначен исполнителем множества заявок через связь AssigneeId.

– При этом один пользователь может выступать как создателем заявок, так и их исполнителем.

– Каждая заявка обязательно имеет одного создателя, но может не иметь исполнителя (AssigneeId допускает NULL).

Отношение «один-ко-многим» между оборудованием и заявками.

– Один тип оборудования может использоваться в множестве заявок.

– Каждая заявка относится к одному конкретному типу оборудования.

– При удалении оборудования требуется каскадная обработка связанных заявок.

Отношение «один-ко-многим» между объектами и заявками.

– Один объект может быть указан в множестве заявок.

– Каждая заявка привязана к одному конкретному объекту.

– Объекты представляют физические местоположения, где установлено оборудование КИПиА.

Отношение "один-ко-многим" между пользователями и токенами обновления.

– Один пользователь может иметь несколько активных токенов обновления.

– Каждый токен привязан к одному конкретному пользователю.

– Токены автоматически помечаются как отозванные (IsRevoked) при выходе из системы.

Отношение «один-ко-многим» между пользователями и сессиями.

– Один пользователь может иметь несколько активных сессий на разных устройствах

– Каждая сессия принадлежит одному пользователю.

– Сессии содержат техническую информацию о подключении (IP-адрес, браузер)

Связи на основе ролевой модели пользователей:

– пользователи с ролью Applicant могут только создавать заявки;

– пользователи с ролью Executor могут быть назначены исполнителями;

– пользователи с ролью Dispatcher могут создавать заявки и назначать исполнителей;

– пользователи с ролью Admin имеют полный доступ ко всем операциям.

5.2.3 Данные в таблице.

	Id [PK] integer	Name text	Description text
1	1	Датчик давления	Измерение давления в трубопроводе
2	2	Контроллер автоматики	Управление системами КИПиА
3	3	Прибор измерения расхода	Контроль расхода газа

Рисунок 5.2 – Таблица Equipments

	Id [PK] integer	Name text	Location text
1	1	ГРС-1	Воткинск, ул. Центральная
2	2	Компрессорная станция-5	Чайковский, район Промзона
3	3	Газопроводный участок-10	Межрегиональный

Рисунок 5.3 – Таблица Objects

5.2.4 ERD – диаграмма

На диаграмме физически установим взаимосвязи между таблицами в базе данных.

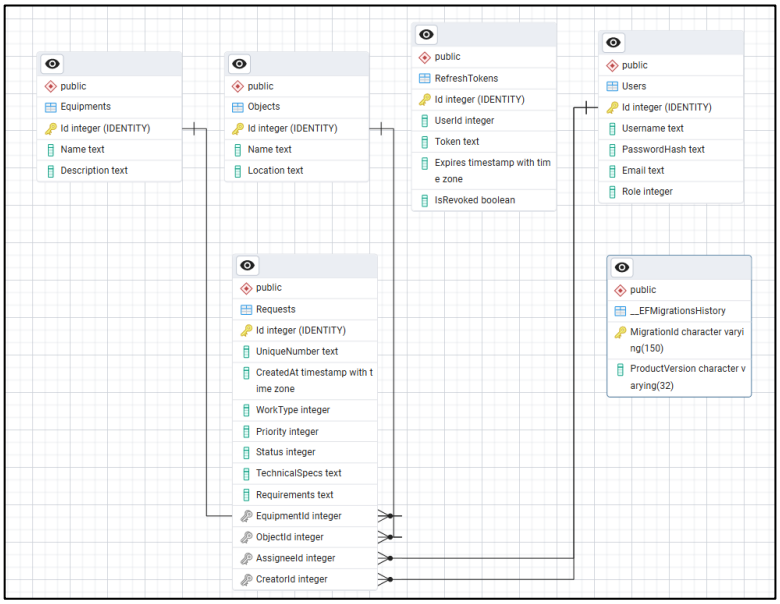

Рисунок 5.4 – ERD – диаграмма

5.3 Практическая реализация приложения

5.3.1 Контроллеры API системы управления заявками КИПиА

1. Контроллер аутентификации (AccountController)

Реализованные методы API:

– POST /api/Account/Login (назначение: аутентификация пользователя в системе);

– POST /api/Account/Register (назначение: регистрация нового пользователя;

– POST /api/Account/Refresh (назначение: обновление access-токена с помощью refresh-токена).

2. Контроллер оборудования (EquipmentsController)

Реализованные методы API:

– GET /api/Equipments (назначение: получение списка всего оборудования);

– GET /api/Equipments/{id}(назначение: получение оборудования по идентификатору).

2. Контроллер объектов (ObjectsController)

Реализованные методы API:

– GET /api/Objects (назначение: получение списка всех объектов);

– GET /api/Objects/{id} (назначение: получение объекта по идентификатору).

4. Контроллер заявок (RequestsController)

Реализованные методы API:

– POST /api/Requests (назначение: создание новой заявки);

– GET /api/Requests (назначение: получение заявок с фильтрацией и сортировкой);

– PUT /api/Requests/{id}/status (назначение: изменение статуса заявки);

– PUT /api/Requests/{id} (назначение: редактирование заявки);

– DELETE /api/Requests/{id}(назначение: удаление заявки);

– PUT /api/Requests/{id}/assign (назначение: назначение исполнителя на заявку).

5. Контроллер пользователей (UsersController)

Реализованные методы API:

– GET /api/Users (назначение: получение списка пользователей).

6. Дополнительные функции системы.

Система ролей:

– Applicant – заявитель (может создавать и редактировать свои заявки);

– Executor – исполнитель (может менять статусы заявок);

– Dispatcher – диспетчер (полный доступ к заявкам, назначение исполнителей);

– Admin – администратор (полные права на управление системой).

Email-уведомления: система автоматически отправляет уведомления при:

– изменении статуса заявки;

– назначении исполнителя;

– создании новых заявок.

Безопасность:

– Jwt –аутентификация;

– ролевая модель доступа;

– refresh tokens для продления сессий;

– защита от несанкционированного редактирования.

Создаем проект формата ASP.NET (MVC) в Visual Studio 2022, первым шагом создадим все необходимые модели в папке Models для с которыми будет происходить взаимодействия с Базой данных, реализация моделей приведена в листинге 5.1.

Листинг 5.1 – Модель User

```
public class User
{
    public int Id { get; set; }
    public string Username { get; set; }
    public string PasswordHash { get; set; } // Шифровать (используйте BCrypt)
    public string Email { get; set; } // Для уведомлений
    public UserRole Role { get; set; } // Enum: Applicant, Executor, Dispatcher, Admin
    public ICollection<Request> AssignedRequests { get; set; }
}
```

По аналогии с моделью Employee реализованы оставшиеся модели.

5.3.2 Реализация контекста БД

Так как в стеке технологий указано, что для взаимодействия с базой данных используется технология EntityFrameworkCore, создадим файл контекста нашей БД, в котором будут прописаны все используемые модели, а так же указаны настройки для взаимодействия таблиц между собой, за счет миграции это поможет менять структуру БД без опасности.

После того как создали все модели и создали их контекст с настройками, создадим контроллеры (AccountController, EnployeeController, BranchController) которые будут реализовывать функционал нашего приложения.

5.4 Инструкция по работе с программой

При запуске приложения открывается экран авторизации. Для входа требуется ввести имя и пароль, затем нажать кнопку «Войти». При отсутствии регистрации необходимо нажать кнопку «Зарегистрироваться».

После этого открывается форма для ввода данных: имя, пароль и выбор роли, соответствующей должности на работе. Завершает процесс регистрации нажатие кнопки «Зарегистрировать».

Успешная регистрация возвращает на экран авторизации. Ввод сохраненных имени и пароля с последующим нажатием кнопки «Войти» выполняет вход в систему. Успешная авторизация подтверждается системным уведомлением. (Рисунок 5.5– 5.7).

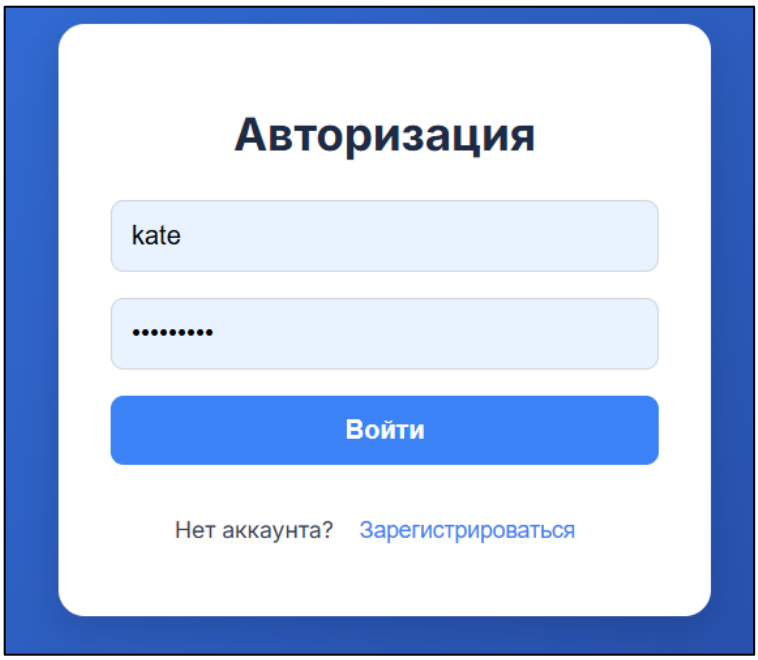

Рисунок 5.5 – Страница «Авторизация»

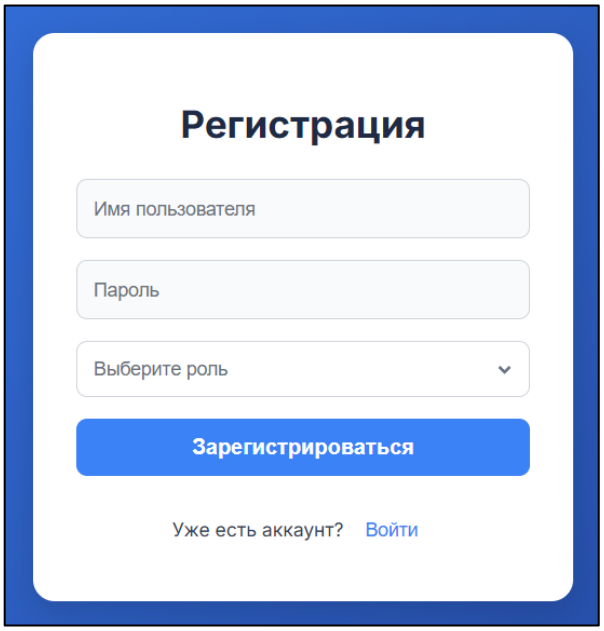

Рисунок 5.6 – Страница «Регистрация»

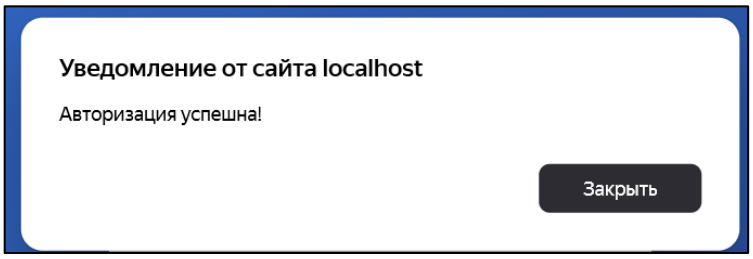

Рисунок 5.7 – Уведомление об успешной авторизации

На рисунке 5.8 представлена панель управления администратора. Интерфейс позволяет управлять заявками: удалять, редактировать, изменять их статус и назначать ответственных исполнителей.

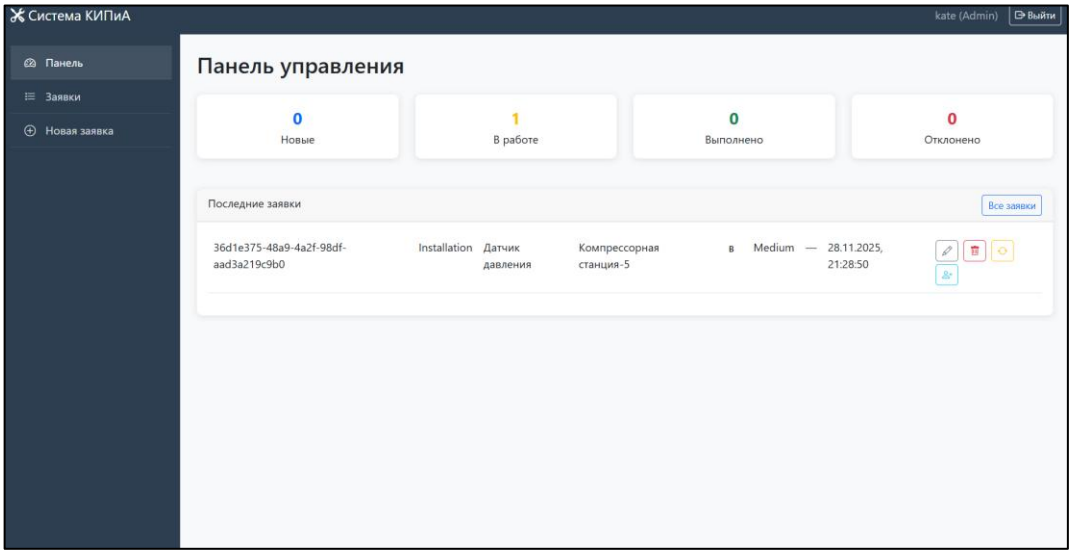

Рисунок 5.8 – Страница «Панель управления»

Процедура удаления заявки: для удаления заявки администратору необходимо нажать на значок корзины, который является элементом интерфейса для выполнения данной операции. После этого действие требует подтверждения в диалоговом окне. После подтверждения заявка окончательно удаляется из системы (Рисунок 5.9 – 5.10).

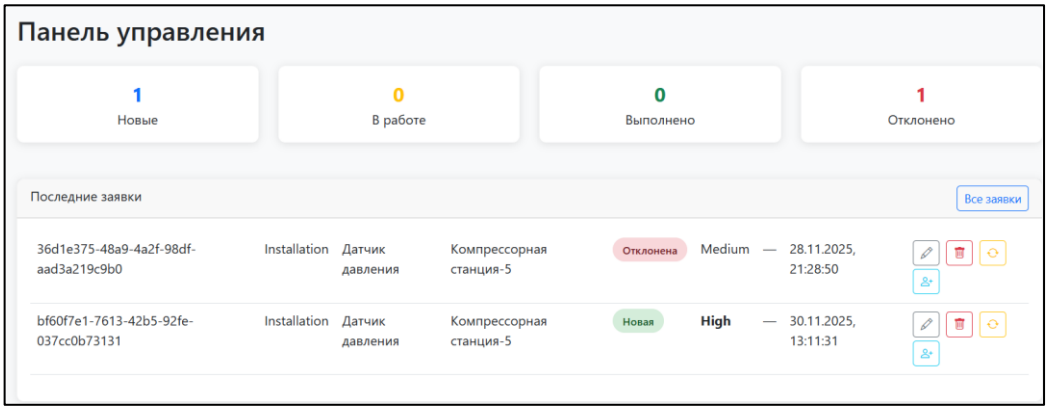

Рисунок 5.9 – До удаления заявки

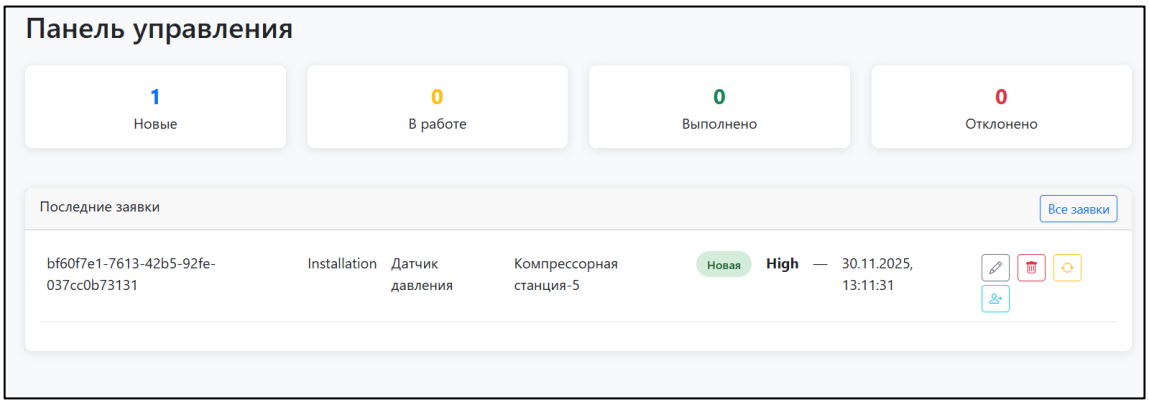

Рисунок 5.10 – После удаления заявки

На рисунке 5.11 отображен раздел с историей заявок, где представлен реестр всех поданных обращений.

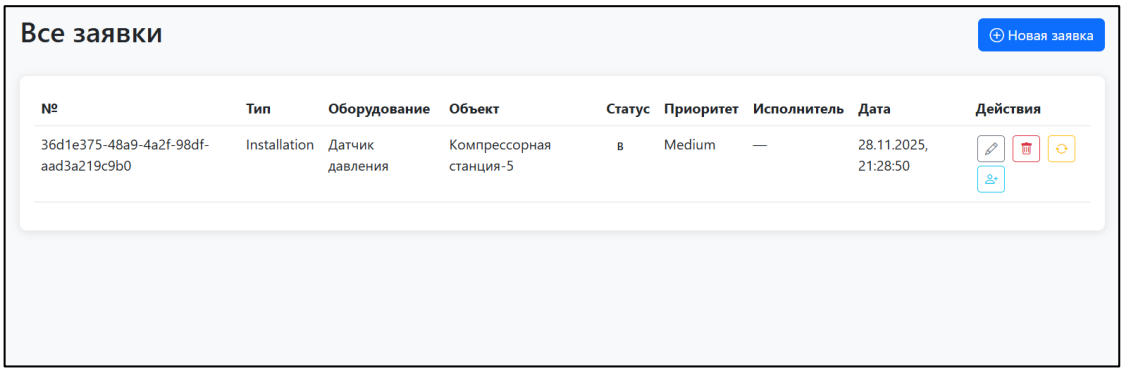

Рисунок 5.11 – Страница «Все заявки»

На рисунке 5.12 показан интерфейс подачи новой заявки. Для создания заявки пользователю необходимо заполнить все обязательные поля формы.

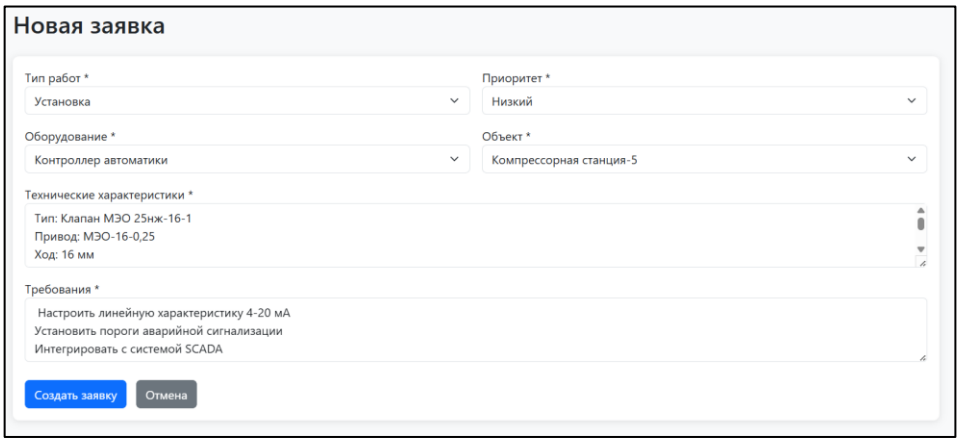

Рисунок 5.12 – Страница «Новая заявка»

Оповещение о новой заявке автоматически направляется администратору по электронной почте. Система уведомлений обеспечивает контроль всех действий пользователей, рассылая на почту администратора информацию о каждом произведенном действии в системе. сделай этот текст для инструкции программы (Рисунок 5.13).

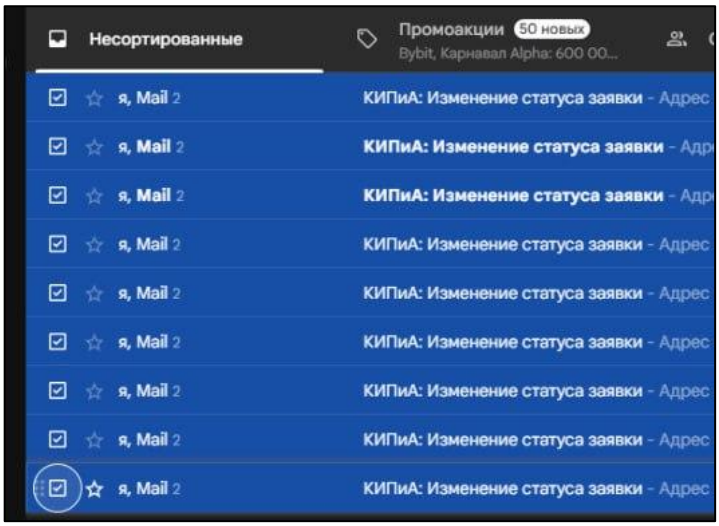

Рисунок 5.13 – Уведомлении на почту

Заключение

Разработанная подсистема представляет собой законченное программное решение для учета продукции, выполненное в соответствии с заданием демонстрационного экзамена.

В процессе работы была спроектирована реляционная база данных (3НФ), в которую успешно импортированы данные из внешних источников. Серверная часть, построенная на ASP.NET Core Web API и Entity Framework Core, обеспечивает выполнение всех необходимых операций: расчет стоимости продукции на основе материалов, а также полноценное управление (создание, чтение, обновление, удаление) как номенклатурой продукции, так и справочником материалов. В коде применены современные практики: асинхронное программирование.

Пользовательский интерфейс, реализованный средствами HTML, CSS и JavaScript, предоставляет удобный доступ ко всем функциям системы. При разработке интерфейса учитывались требования к стилевому оформлению, навигационной структуре и обработке пользовательских ошибок.

Итоговый продукт в полном объеме (исходные коды, скрипты базы данных, исполняемые файлы) зафиксирован в системе контроля версий.

Список использованных источников

1 Кода генерация DTO: зачем нужна и как её настроить [электронный ресурс] – URL: https://habr.com/ru/companies/yandex/articles/598125/

2 Руководство по ASP.NET Core 8 [электронный ресурс] – URL: https://metanit.com/sharp/aspnet6/

3 Работа с данными в Entity Fraemwoek [электронная среда] – URL: https://metanit.com/sharp/aspnetmvc/11.2.php

4 Формат JSON, метод toJSON [электронный ресурс] – URL: https://learn.javascript.ru/json

5 Основы JavaScript [электронный ресурс] – URL: https://learn.javascript.ru/first-steps

6 База по вёрстке [электронный ресурс] – URL: https://thecode.media/baza-po-vyorstke-osnovy-html/